

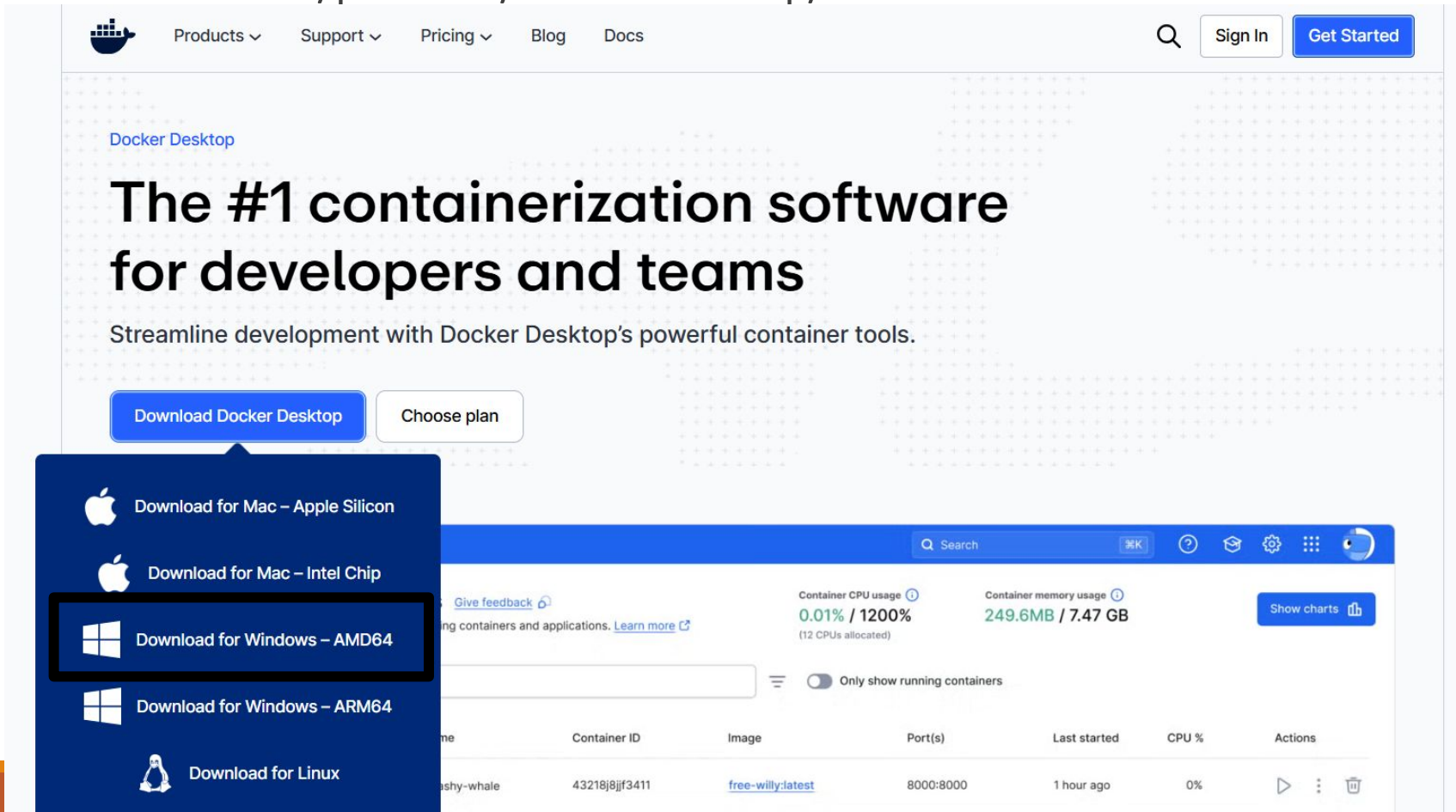
Aplicações Docker

MSC JOSE ALFREDO MENDOZA PEÑALOZA

16/06/2026

Baixar Docker

<https://www.docker.com/products/docker-desktop/>



The screenshot shows the Docker Desktop download page. The main heading is "The #1 containerization software for developers and teams". Below it, a subheading reads "Streamline development with Docker Desktop's powerful container tools." There are two buttons: "Download Docker Desktop" and "Choose plan". A dropdown menu is open from the "Download Docker Desktop" button, showing five options: "Download for Mac – Apple Silicon", "Download for Mac – Intel Chip", "Download for Windows – AMD64" (which is highlighted with a red box), "Download for Windows – ARM64", and "Download for Linux". Below the menu, there is a section showing container usage statistics: "Container CPU usage 0.01% / 1200% (12 CPUs allocated)" and "Container memory usage 249.6MB / 7.47 GB". At the bottom, there is a table of running containers.

Container ID	Image	Port(s)	Last started	CPU %	Actions
43218j8jf3411	free-willy:latest	8000:8000	1 hour ago	0%	  

Instalação versão do sistema Windows linux

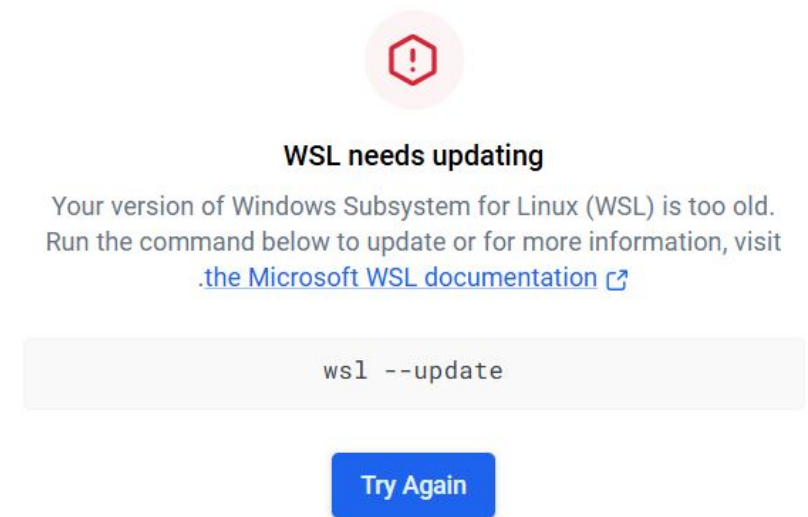
Se a seguinte mensagem aparecer no Docker, você deve instalar uma versão mais atualizada do Linux para Windows, pois muitos sistemas operacionais Windows não o incluem. Ele é necessário para a comunicação com o kernel do sistema operacional.

Precisamos abrir o comando do Windows (cmd) como administrador para este processo.

use os seguintes comandos

```
wsl --update
```

A seguir o sistema precisa ser reiniciado.



Aba de contêineres Docker

Containers

Images

Logs

Volumes

Kubernetes

Builds

Models

Docker Hub

MCP Toolkit BETA

Extensions

Container CPU usage ⓘ
No containers are running.

Container memory usage ⓘ
No containers are running.

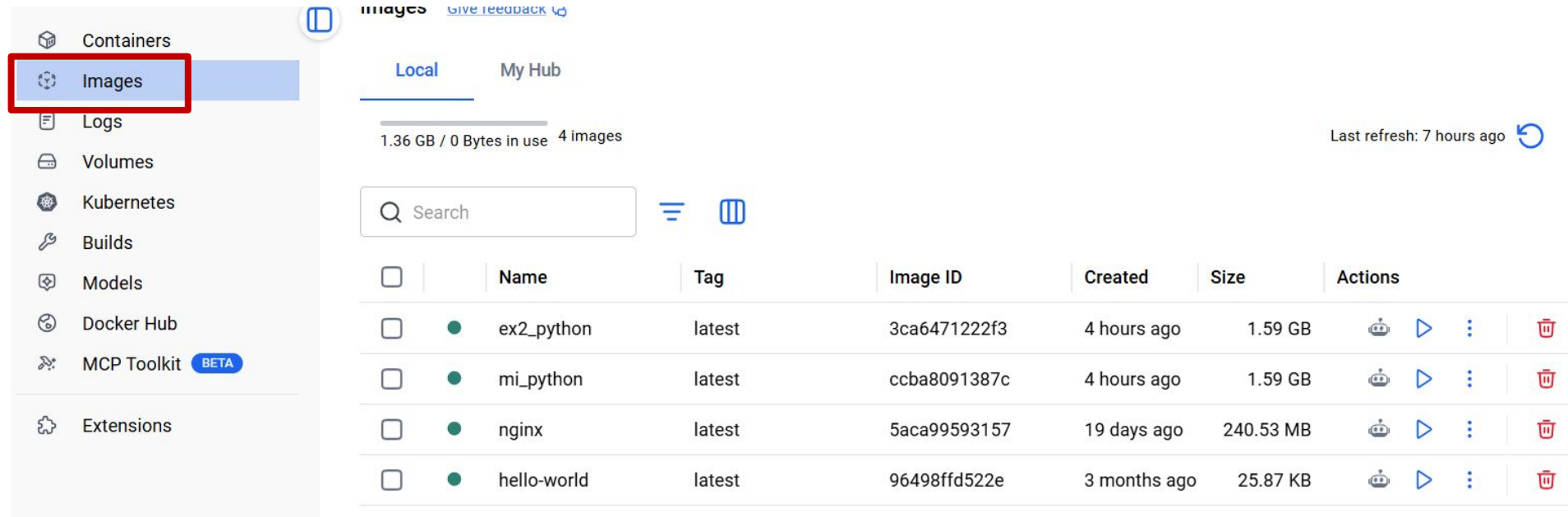
Show charts

Search

Only show running containers

☐		Name	Container ID	Image	Port(s)	CPU (%)	Actions
☐	☐	infallible_jones	201da055d830	ex2_pythor		N/A	   
☐	☐	crazy_wu	a705959b3e57	mi_python		N/A	   
☐	☐	mi_nginx	e97b80bcd970	nginx	8080:80	N/A	   
☐	☐	boring_chandrasekhar	8359e18b5ee8	hello-world		N/A	   

Aba de imagens do Docker



The screenshot displays the Docker Desktop interface, specifically the 'Images' tab. The left sidebar contains a list of navigation items: Containers, Images (highlighted with a red box), Logs, Volumes, Kubernetes, Builds, Models, Docker Hub, MCP Toolkit (marked BETA), and Extensions. The main content area shows the 'Local' tab for images. It indicates that 1.36 GB / 0 Bytes are in use and there are 4 images. A search bar and filter icons are present. Below is a table listing the images:

☐		Name	Tag	Image ID	Created	Size	Actions
☐	●	ex2_python	latest	3ca6471222f3	4 hours ago	1.59 GB	
☐	●	mi_python	latest	ccba8091387c	4 hours ago	1.59 GB	
☐	●	nginx	latest	5aca99593157	19 days ago	240.53 MB	
☐	●	hello-world	latest	96498ffd522e	3 months ago	25.87 KB	

Hello-world no docker

Entramos novamente no cmd do Windows e digitamos o seguinte comando.

```
docker run hello-world
```

Uma mensagem como a seguinte será exibida.

```
Hello from Docker!  
This message shows that your installation appears to be working correctly.  
  
To generate this message, Docker took the following steps:  
1. The Docker client contacted the Docker daemon.  
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.  
   (amd64)  
3. The Docker daemon created a new container from that image which runs the  
   executable that produces the output you are currently reading.  
4. The Docker daemon streamed that output to the Docker client, which sent it  
   to your terminal.  
  
To try something more ambitious, you can run an Ubuntu container with:  
$ docker run -it ubuntu bash  
  
Share images, automate workflows, and more with a free Docker ID:  
https://hub.docker.com/  
  
For more examples and ideas, visit:  
https://docs.docker.com/get-started/
```

Hello-world no docker

Além disso, o contêiner hello-word aparecerá nas vendas de contêineres Docker.

Search

Only show running containers

Isso confirma que o Docker está funcionando corretamente, baixando imagens para criar os contêineres.

Docker com Nginx

Em seguida, faremos o download do nginx para criar servidores web. Usamos o seguinte comando para baixar a imagem do nginx.

```
docker pull nginx
```

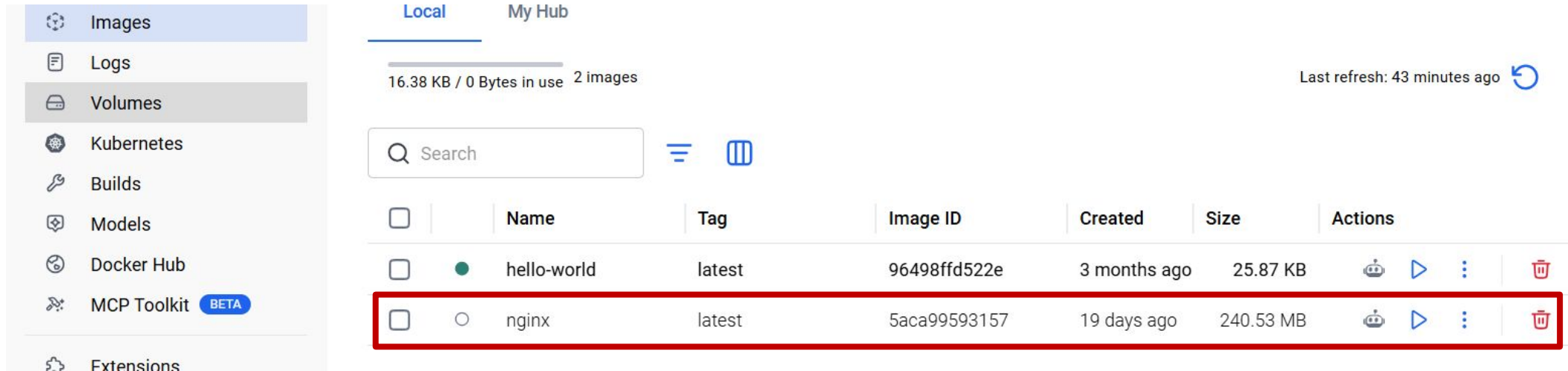
As informações de download aparecem.

```
C:\WINDOWS\system32>docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
13fd728be9eb: Pull complete
5431d0092fffd: Pull complete
7f8b1a2b17d8: Pull complete
830625e1ac85: Pull complete
45381ecb0e2f: Pull complete
5b4d6ff92fc4: Pull complete
b4a248c845e5: Pull complete
cc5f57206478: Download complete
5e6b66b5e5f1: Download complete
Digest: sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview nginx
```


Docker com Nginx

Acesse a aba de imagens para confirmar o download da imagem do nginx.



The screenshot shows the Docker Desktop interface. On the left is a sidebar with navigation options: Images (selected), Logs, Volumes, Kubernetes, Builds, Models, Docker Hub, MCP Toolkit (BETA), and Extensions. The main area displays the 'Local' tab for images. It shows a search bar, a refresh button, and a table of local images. The table has columns for Name, Tag, Image ID, Created, Size, and Actions. Two images are listed: 'hello-world' and 'nginx'. The 'nginx' row is highlighted with a red border. Above the table, it indicates '16.38 KB / 0 Bytes in use' and '2 images'. A refresh status shows 'Last refresh: 43 minutes ago'.

	Name	Tag	Image ID	Created	Size	Actions
☐	hello-world	latest	96498ffd522e	3 months ago	25.87 KB	
☐	nginx	latest	5aca99593157	19 days ago	240.53 MB	

Docker com Nginx

Também podemos usar o cmd com o comando "docker images" para visualizar as imagens que foram baixadas para o nosso contêiner Docker.

```
C:\WINDOWS\system32>docker imagenes
docker: unknown command: docker imagenes

Run 'docker --help' for more information

C:\WINDOWS\system32>docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	96498ffd522e	25.9kB	9.49kB	U
nginx:latest	5aca99593157	241MB	66MB	

Docker com Nginx

Em seguida, executamos o contêiner nginx usando o seguinte comando.

```
docker run -d -p 8080:80 --name mi_nginx nginx
```

Onde:

-d executa o contêiner em segundo plano.

-p 8080:80 conecta a porta 8080 do Windows à porta 80 do contêiner.

--name my_nginx atribui um nome ao contêiner.

nginx é a imagem utilizada.

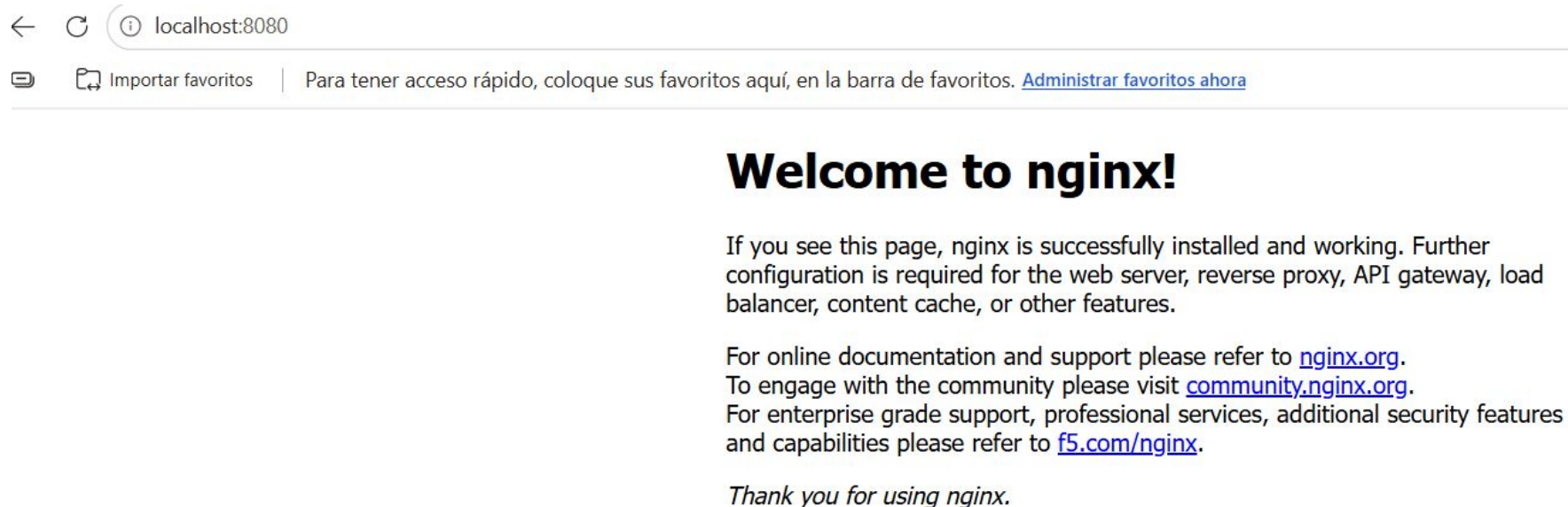
```
C:\WINDOWS\system32>docker run -d -p 8080:80 --name mi_nginx nginx  
e97b80bcd970df1d0cc4149a8df96bcc897ce9c39aff24e8d6bdc93b4b303dd3
```

Docker com Nginx

Em seguida, acesse o navegador e use o seguinte link.

```
http://localhost:8080
```

A página de Bem-vindo do nginx é exibida, onde podemos começar a modificá-la a partir do código do contêiner nginx.



Configuração do Docker

Com o comando "docker ps" podemos ver os contêineres que estão em execução, além de algumas informações como a imagem, a porta de rede e a data de criação.

```
C:\WINDOWS\system32>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
NAMES
e97b80bcd970   nginx    "/docker-entrypoint...." 5 minutes ago  Up 5 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
mi_nginx
```

Comandos Docker

- comando para iniciar o contêiner

```
docker start mi_nginx
```

```
docker start -a crazy_wu
```

- comando para parar o contêiner

```
docker stop mi_nginx
```

- comando para remover contêiner

```
docker rm mi_nginx
```

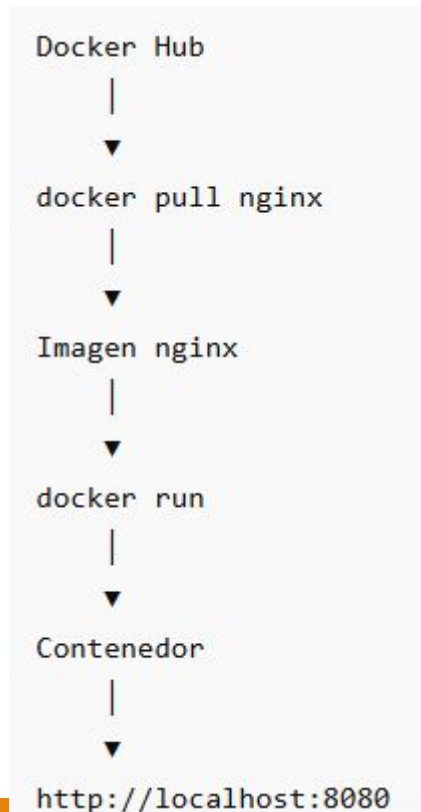
Comandos Docker

- comando para excluir imagem

```
docker rmi nginx
```

Resumo da criação de um contêiner

Para criar uma aplicação, precisamos baixar a imagem do software desejado, criar um dockfile para a criação do contêiner e, em seguida, após a criação do contêiner, podemos executar as aplicações desejadas.



Exemplo usando Docker e Python

Criamos um diretório para a aplicação Python usando os seguintes comandos.

```
mkdir docker_python  
cd docker_python
```

Em seguida, criamos um aplicativo Python com o seguinte código.

```
print("Hello-world desde Docker e Python")
```

Exemplo usando Docker e Python

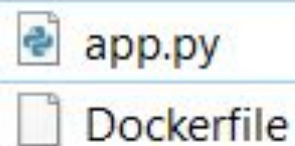
Em seguida, criamos um arquivo chamado Dockerfile, que fornecerá as instruções para a criação do contêiner, com o seguinte código.

```
FROM python:3.12
```

```
WORKDIR /app
```

```
COPY app.py .
```

```
CMD ["python", "app.py"]
```



Explicação do Dockerfile

Explicação:

FROM python:3.12: Usa uma imagem oficial do Python.

WORKDIR /app: Define o diretório de trabalho.

COPY app.py .: Copia o arquivo para o contêiner.

CMD: Executa o programa quando o contêiner é iniciado.

Exemplo usando Docker e Python

Em seguida, na pasta do projeto, usamos o seguinte comando:

```
docker build -t mi_python .
```

inicia o processo de instalação.

```
C:\python_docker>docker build -t my_python .
[+] Building 27.8s (5/8)
=> [internal] load build definition from dockerfile
=> => transferring dockerfile: 114B
=> [internal] load metadata for docker.io/library/python:3.12
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/python:3.12@sha256:bd55e06128e2743d526738dd5cf5db924e49ead66d4fe2ad15f7
=> => resolve docker.io/library/python:3.12@sha256:bd55e06128e2743d526738dd5cf5db924e49ead66d4fe2ad15f7
=> => sha256:e27d97337704ffab0806d3a79ee0b46489fb2faf990734064910f0fba0ee2929 250B / 250B
=> => sha256:8564ac3925855f99fe8eed20231ce40c61bdc19f2f83ca9520cf798fcb252baf 25.72MB / 25.72MB
=> => sha256:b6fc5908f1d6112c1149d97b4e71973c165c034761656a20667d6f879055ca58 6.09MB / 6.09MB
=> => sha256:8d6d44b254dab2063c4226fc8a0849d5527402d24d3bea80d644a1e4ac3a47e5 170.92MB / 236.18MB
=> => sha256:b53089dca50590292ecc77bf803152a5799650e734717e4b706cb812a02073ba 67.78MB / 67.78MB
=> => sha256:8a7504cd2818ce40ac76c17886a03dfff25ef0aa06fff6125bf0f0c7302cdc6471 25.63MB / 25.63MB
=> => sha256:f32f49ce655a9cf7c1fd4ca1417ddb39a54cedf4b7ff35de20f8009c18dd7a96 49.31MB / 49.31MB
=> => extracting sha256:f32f49ce655a9cf7c1fd4ca1417ddb39a54cedf4b7ff35de20f8009c18dd7a96
=> => extracting sha256:8a7504cd2818ce40ac76c17886a03dfff25ef0aa06fff6125bf0f0c7302cdc6471
=> => extracting sha256:b53089dca50590292ecc77bf803152a5799650e734717e4b706cb812a02073ba
=> [internal] load build context
=> => transferring context: 74B
```

Exemplo usando Docker e Python

Confirmamos as imagens criadas pelo Docker com os diretórios Confirmamos as imagens criadas pelo Docker com os diretórios correspondentes através do comando "docker images".

```
C:\python_docker>docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	96498ffd522e	25.9kB	9.49kB	U
mi_python:latest	ccba8091387c	1.6GB	411MB	
my_python:latest	489b998fef61	1.6GB	411MB	
nginx:latest	5aca99593157	241MB	66MB	U

 Info

Exemplo usando Docker e Python

Em seguida, executamos o seguinte comando.

```
docker run mi_python
```

Após executar o projeto, obtivemos o seguinte resultado.

```
C:\python_docker>docker run ex1_python  
Hello-world desde Docker e Python
```

Exemplo 2: python com docker

Criamos um novo diretório com dois arquivos, app.py e Dockerfile. O Dockerfile terá o mesmo conteúdo de antes, e o aplicativo Python terá o seguinte código.

```
1 nome = input("Insira seu nome: ")
2
3 print(f"Bem-vindo {nome}")
4 print("Programa executado através do Docker")
```

Exemplo 2: python com docker

Em seguida, criamos a nova imagem do Python usando o seguinte comando.

```
docker build -t mi_python .
```

Em seguida, executamos o contêiner com o seguinte comando.

```
docker run -it mi_python
```


Exemplo 2: python com docker

O segundo exemplo envolve registrar o nome do usuário em uma variável e, em seguida, imprimi-lo.

Obtemos o seguinte resultado

```
C:\docker_python2>docker run -it ex2_python  
Insira seu nome: jose  
Bem-vindo jose  
Programa executado através do Docker
```